RAG-Based Customer Support Assistant using LangGraph and HITL

Technical Documentation & Implementation Guide

This document serves as a comprehensive guide to implementing a cutting-edge customer support solution, leveraging Retrieval-Augmented Generation (RAG) and Human-In-The-Loop (HITL) workflows to automate and enhance customer interactions.

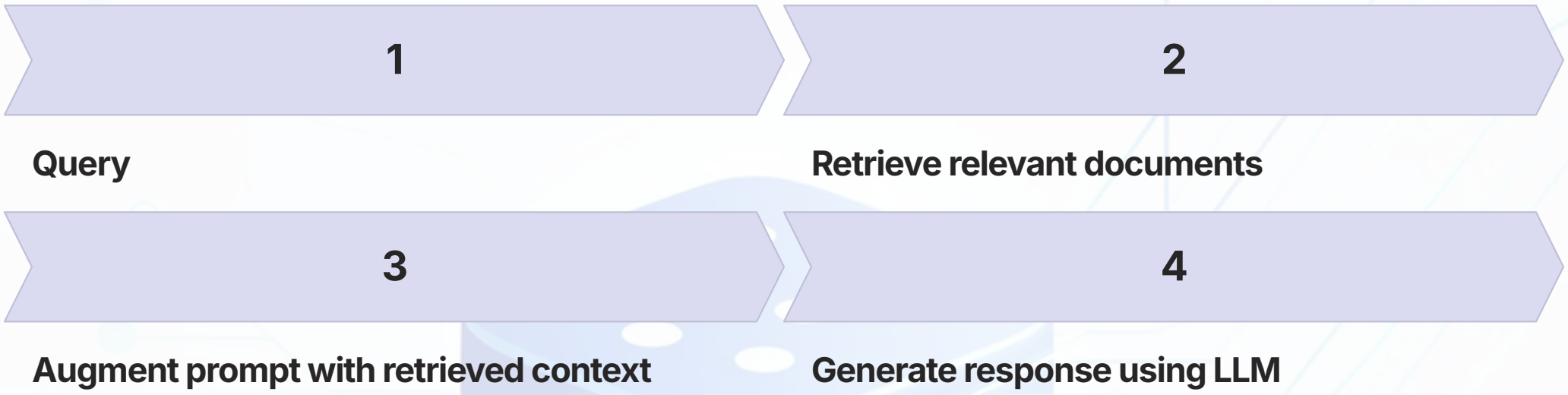
Made with GAMMA

1. Introduction

What is RAG (Retrieval-Augmented Generation)?

RAG is a hybrid approach that combines information retrieval with generative AI. Instead of relying solely on an LLM's training data, RAG systems retrieve relevant documents from a knowledge base and use them as context for generating responses.

The workflow is:



Why RAG is Needed

Traditional LLMs have several limitations in customer support:

- Knowledge cutoff dates (training data becomes stale)
- Hallucination risk (generating plausible but incorrect information)
- No access to company-specific documentation or policies
- Inability to cite sources or provide verifiable answers

RAG solves these by grounding responses in actual company documents, ensuring accuracy and traceability.

Use Case: Customer Support Automation

This system automates first-line customer support by:

- Instantly retrieving relevant documentation for incoming queries
- Generating contextually accurate responses without hallucination
- Escalating complex issues to human agents via HITL
- Maintaining conversation history for context-aware follow-ups
- Reducing response time from hours to seconds while maintaining quality

Made with GAMMA

2. System Architecture Explanation

Core Components

1	**Document Ingestion Pipeline** • Loads customer support documentation (PDFs, markdown, text files) • Splits documents into chunks (400 tokens with 50-token overlap) • Generates embeddings for each chunk using a pre-trained model • Stores embeddings and metadata in ChromaDB
2	**Vector Database (ChromaDB)** • Stores document chunks and their embeddings • Enables semantic similarity search • Retrieves top-k most relevant chunks for a given query • Lightweight, in-memory or persistent storage options
3	**Embedding Model** • Converts text (queries and documents) into numerical vectors • Enables semantic similarity matching • Used: sentence-transformers (all-MiniLM-L6-v2) for efficiency
4	**LLM (LLaMA via Ollama)** • Generates contextually relevant responses • Runs locally for privacy and cost efficiency • Receives augmented prompts with retrieved context • Supports streaming for real-time response generation
5	**LangGraph Orchestration** • Manages workflow state and transitions • Defines nodes for each processing step • Implements conditional routing logic • Handles HITL escalation workflows
6	**State Management** • Maintains conversation history • Tracks query, retrieved documents, generated response • Stores escalation flags and human feedback • Enables multi-turn conversations

Component Interactions

The system follows this flow:

This detailed workflow illustrates the sequential steps and decision points within the RAG-based customer support assistant, highlighting the integration of AI components with human intervention for optimal performance.

Made with GAMMA

3. Design Decisions

Chunk Size: 400 Tokens with 50-Token Overlap

Why 400 tokens?

- Balances context preservation with retrieval precision
- ~300 words per chunk, sufficient for coherent information units
- Smaller chunks (200 tokens) lose context; larger chunks (800+) dilute relevance
- Fits within most LLM context windows with room for multiple retrieved chunks

Why 50-token overlap?

- Prevents information loss at chunk boundaries
- Ensures concepts spanning chunk edges remain retrievable
- Minimal redundancy overhead (~12.5% duplication)
- Improves semantic continuity in multi-chunk responses

Embedding Model: sentence-transformers (all-MiniLM-L6-v2)

Selection rationale:

- Lightweight (22M parameters) vs. larger models (335M+)
- Fast inference (~10ms per query) suitable for real-time support
- Strong semantic understanding despite smaller size
- Pre-trained on 215M sentence pairs, optimized for similarity tasks
- Open-source, runs locally without API dependencies
- Sufficient for customer support domain (not specialized NLP tasks)

Trade-off: Slightly lower accuracy than larger models (e.g., all-mpnet-base-v2) but acceptable for support queries where relevance is more important than nuance.

Vector Database: ChromaDB

Why ChromaDB over alternatives?

- Lightweight and easy to integrate (no separate server needed)
- Built-in embedding support (can use any embedding model)
- Persistent storage option for production deployments
- Efficient similarity search with configurable distance metrics
- Supports metadata filtering (e.g., filter by document type or date)

Limitations acknowledged:

- Not suitable for billion-scale datasets (use Pinecone, Weaviate for scale)
- Single-machine deployment (no distributed clustering)
- Acceptable for typical customer support knowledge bases (10K-100K documents)

LLM Choice: LLaMA via Ollama

Why local LLM instead of API-based (GPT-4, Claude)?

- Privacy: Customer queries never leave the organization
- Cost: No per-token charges; one-time model download
- Latency: Sub-second response times (no network round-trips)
- Control: Full customization of model behavior and fine-tuning

LLaMA 2 (7B or 13B variant) selection:

- 7B: Fast inference, suitable for real-time support (recommended)
- 13B: Better reasoning, handles complex queries (if hardware allows)
- Ollama simplifies deployment (single command: `ollama run llama2`)

Trade-off: Slightly lower quality than GPT-4 but acceptable for support tasks where responses are grounded in retrieved documents.

Prompt Design Strategy

System prompt structure:

```
You are a helpful customer support assistant. Answer questions based ONLY on the provided context.
If the context doesn't contain the answer, say "I don't have information about this."
Be concise, professional, and cite the relevant section when possible.

Context:
[RETRIEVED_CHUNKS_HERE]

User Question: [USER_QUERY]
```

Key principles:

- Explicit instruction to use only provided context (reduces hallucination)
- Permission to admit knowledge gaps (builds trust)
- Encouragement to cite sources (improves traceability)
- Conciseness requirement (prevent verbose responses)
- Role definition (sets professional tone)

Made with GAMMA

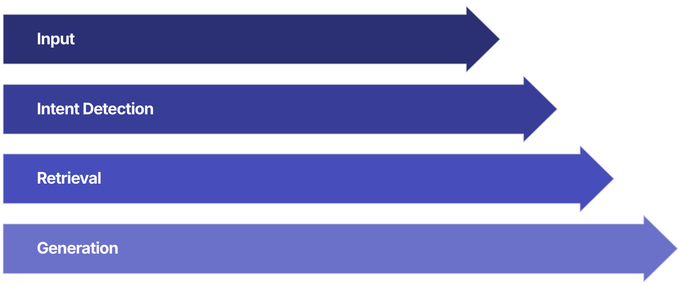
4. Workflow Explanation: LangGraph Implementation

LangGraph Overview

LangGraph is a framework for building stateful, multi-step AI workflows. It models workflows as directed graphs where:

- **Nodes** represent processing steps (functions)
- **Edges** represent transitions between nodes
- **State** is passed and updated through the graph
- **Conditional edges** route execution based on state values

Graph Structure for This System



Node Responsibilities

1	**Input Node** • Receives user query and conversation history • Initializes state with: query, conversation_history, retrieved_docs, response, confidence_score, escalated • Validates input (non-empty query, reasonable length)
2	**Intent Detection Node** • Analyzes query to classify intent (e.g., "billing", "technical", "account", "general") • Uses simple keyword matching or lightweight classifier • Updates state: intent, intent_confidence • Purpose: Enables future routing to specialized handlers
3	**Retrieval Node** • Embeds user query using sentence-transformers • Queries ChromaDB for top-k similar chunks (k=3-5) • Retrieves metadata (source document, section, date) • Updates state: retrieved_docs, retrieval_score • Handles edge case: If no relevant documents found, sets flag for HITL escalation
4	**Generation Node** • Constructs augmented prompt: system_prompt + retrieved_context + user_query • Calls LLaMA via Ollama with temperature=0.3 (low randomness for consistency) • Streams response for real-time feedback • Updates state: response, generation_timestamp • Includes error handling for LLM failures

5	**Confidence Check Node**
	• Evaluates response quality using multiple signals:
	  ◦ Retrieval score (how relevant were the documents?)
	  ◦ Response length (too short might indicate insufficient context)
	  ◦ Presence of uncertainty phrases ("I don't know", "not sure")
	  ◦ Intent-response alignment (does response match the intent?)
	• Calculates confidence_score (0.0-1.0)
	• Updates state: confidence_score, should_escalate
	• Threshold: If confidence < 0.6, mark for escalation

6	**Output/Escalation Node** • If confidence >= 0.6: Return response to user • If confidence < 0.6: Escalate to HITL with context • Logs interaction for analytics and feedback

State Transitions

State object structure:

```
{
  "query": str,
  "conversation_history": List[Dict],
  "intent": str,
  "intent_confidence": float,
  "retrieved_docs": List[Dict],
  "retrieval_score": float,
  "response": str,
  "confidence_score": float,
  "should_escalate": bool,
  "escalation_reason": str,
  "human_feedback": Optional[str],
  "timestamp": str
}
```

Transition logic:

- Input → Intent Detection (always)
- Intent Detection → Retrieval (always)
- Retrieval → Generation (always, even if no docs found)
- Generation → Confidence Check (always)
- Confidence Check → Output (if confidence >= 0.6)
- Confidence Check → HITL Escalation (if confidence < 0.6)

Error Handling

- If retrieval fails: Continue with empty context, mark for escalation
- If LLM fails: Return error message, escalate to HITL
- If state is corrupted: Log error, restart conversation

Made with GAMMA

5. Conditional Logic: Intent Detection & Routing

Intent Detection Mechanism

The system classifies queries into predefined intents to enable specialized handling:

Intent categories:

Billing
- Payment, invoicing, subscription, refunds

Technical
- Bugs, errors, API issues, integration problems

Account
- Login, password reset, profile management

General
- Product info, features, best practices

Complaint
- Negative feedback, service issues

Unknown
- Doesn't fit above categories

Detection method (hybrid approach):

1. **Keyword matching** (fast, rule-based)
   - Billing keywords: "charge", "invoice", "payment", "refund", "subscription"
   - Technical keywords: "error", "bug", "crash", "API", "integration"
   - Account keywords: "login", "password", "account", "profile"
2. **Fallback to lightweight classifier** (if keywords ambiguous)
   - Uses a small fine-tuned model or zero-shot classification
   - Provides intent_confidence score (0.0-1.0)

Example:

- Query: "Why was I charged twice?"
  - Keywords match: "charged" → Billing intent
  - intent_confidence: 0.95
- Query: "The app keeps crashing when I upload files"
  - Keywords match: "crashing", "upload" → Technical intent
  - intent_confidence: 0.92

Routing Decisions

After intent detection, the system makes routing decisions:

01

Standard Routing

(confidence >= 0.7)

- Proceed to retrieval with intent-specific filters
- Example: For Billing intent, prioritize billing-related documents

02

Uncertain Routing

(confidence 0.5-0.7)

- Retrieve documents from multiple intent categories
- Broader context to handle ambiguity
- Example: Query could be billing or account-related

03

Unknown Intent

(confidence < 0.5)

- Retrieve from general knowledge base
- Flag for potential HITL escalation if response confidence is low

Confidence Scoring Logic

The system calculates confidence_score using weighted factors:

$$confidence_score = (0.3 * retrieval_score + 0.3 * intent_confidence + 0.2 * response_quality_score + 0.2 * context_relevance_score)$$

- **retrieval_score**: Cosine similarity of top retrieved document (0.0-1.0)
  - High (>0.8): Very relevant documents found
  - Medium (0.5-0.8): Somewhat relevant documents
  - Low (<0.5): Weak relevance, likely escalate
- **intent_confidence**: Confidence in intent classification (0.0-1.0)
  - High (>0.8): Clear intent
  - Medium (0.5-0.8): Ambiguous intent
  - Low (<0.5): Unclear intent
- **response_quality_score**: Heuristic evaluation of generated response
  - Checks for uncertainty phrases ("I don't know", "unclear")
  - Evaluates response length (too short = insufficient context)
  - Checks for citations (presence of source references)
  - Score: 0.0-1.0
- **context_relevance_score**: How well retrieved context matches the query
  - Semantic similarity between query and retrieved chunks
  - Presence of key terms from query in retrieved documents
  - Score: 0.0-1.0

Escalation Thresholds

Escalation triggers:

→ **Automatic escalation**

(confidence_score < 0.6)

- Response quality too uncertain
- Human review needed

→ **Intent-based escalation**

- Complaint intent + negative sentiment → Always escalate
- Unknown intent + low retrieval score → Escalate

→ **Content-based escalation**

- Query contains sensitive keywords (payment details, personal info)
- Response requires human judgment (policy exceptions, refunds)

→ **User-initiated escalation**

- User explicitly requests human agent
- User marks response as unhelpful

Example Routing Scenario

User query: "I can't log into my account and I'm being charged for a service I don't use"

1

Intent Detection:

- Keywords: "login" (Account), "charged" (Billing)
- Detected intent: Account (primary), Billing (secondary)
- intent_confidence: 0.78 (ambiguous)

2

Routing Decision:

- Confidence 0.78 → Uncertain routing
- Retrieve from both Account and Billing document categories

3

Retrieval:

- Top 3 docs: Account recovery (0.85), Billing FAQ (0.72), Subscription management (0.68)
- retrieval_score: 0.85

4

Generation:

- LLM generates response addressing both issues
- Suggests account recovery steps and billing review

5

Confidence Check:

- retrieval_score: 0.85
- intent_confidence: 0.78
- response_quality_score: 0.82 (clear, actionable, cites sources)
- context_relevance_score: 0.80
- Final confidence_score: 0.81 (0.3*0.85 + 0.3*0.78 + 0.2*0.82 + 0.2*0.80)

6

Decision:

confidence_score (0.81) >= 0.6 → Return response to user

- However, flag for monitoring due to multi-intent complexity

Made with GAMMA

6. HITL (Human-In-The-Loop) Implementation

Why HITL is Needed

Despite RAG's improvements, certain scenarios require human judgment:

- Complex multi-part queries requiring nuanced understanding
- Sensitive issues (complaints, refunds, policy exceptions)
- Queries outside the knowledge base scope
- Low-confidence responses that could mislead users
- Feedback collection to improve the system over time

HITL ensures the system remains trustworthy while maintaining automation benefits.

When HITL is Triggered

Escalation occurs in these scenarios:

1

Confidence-based escalation

(primary trigger)

- confidence_score < 0.6
- System uncertain about response quality
- Example: Query about obscure feature with no matching documents

2

Intent-based escalation

- Complaint intent detected
- Refund/exception request
- Sensitive personal information involved
- Example: "I want a refund because your service is terrible"

3

Content-based escalation

- Query contains keywords: "refund", "exception", "complaint", "urgent"
- Response would require policy override
- Example: "Can you make an exception to our standard policy?"

4

Retrieval failure escalation

- No relevant documents found (retrieval_score < 0.3)
- System cannot ground response in knowledge base
- Example: Query about unreleased features

5

User-initiated escalation

- User explicitly requests human agent
- User marks AI response as unhelpful
- User requests callback or follow-up

HITL Workflow

When escalation is triggered:

01

Escalation Package Preparation

- Compile: original query, conversation history, retrieved documents, AI response, confidence_score, escalation_reason
- Format for human readability
- Assign priority based on intent (complaint = high, general = low)

02

Queue Management

- Add to HITL queue with timestamp
- Assign to available human agent
- Set SLA based on priority (complaints: 15 min, general: 1 hour)

03

Human Agent Review

- Agent sees full context: query, AI response, retrieved documents
- Agent can:
  - Approve AI response (mark as correct)
  - Modify AI response (improve or correct)
  - Provide custom response (if AI response is inadequate)
  - Escalate further (to specialist, manager)

04

Feedback Collection

- Agent provides feedback: "correct", "partially correct", "incorrect"
- Agent adds notes explaining the decision
- Agent rates response quality (1-5 stars)

05

Response Delivery

- Final response (AI or human-modified) sent to user
- User informed: "Your query was reviewed by our support team"
- Option to provide feedback on human response

06

Feedback Loop

- Store interaction: query, AI response, human feedback, final response
- Use feedback to:
  - Fine-tune confidence thresholds
  - Identify knowledge base gaps
  - Retrain intent classifier
  - Improve prompt design

HITL Benefits

Quality Assurance

- Catches AI errors before reaching users
- Ensures sensitive issues handled appropriately
- Maintains brand reputation

Continuous Improvement

- Human feedback identifies system weaknesses
- Data for retraining and fine-tuning
- Insights into customer needs and pain points

User Trust

- Users know complex issues get human attention
- Transparency about AI limitations
- Personalized touch for important queries

Operational Insights

- Identifies common escalation patterns
- Reveals knowledge base gaps
- Highlights frequently misunderstood topics

Metrics & Monitoring

Key HITL metrics:

- **Escalation Rate**: % of queries escalated to HITL
  - Target: 15-25% (too low = system overconfident; too high = system underutilized)
- **Human Approval Rate**: % of escalated queries where human approves AI response
  - Target: 60-70% (indicates good AI performance)
- **Average Resolution Time**: Time from escalation to human response
  - Target: < 15 min for complaints, < 1 hour for general
- **Feedback Quality**: % of escalations with actionable feedback
  - Target: > 80%
- **System Improvement Rate**: % of feedback used to improve system
  - Track: How many knowledge base updates, prompt improvements, threshold adjustments

Example HITL Scenario

User query: "I was overcharged and I'm very upset. I want a full refund immediately."

1

System Processing

- Intent: Complaint + Billing
- Retrieval: Finds refund policy (0.72 relevance)
- Generation: "According to our refund policy, refunds are available within 30 days..."
- Confidence: 0.55 (low due to emotional tone and refund request)

2

Escalation Trigger

- confidence_score (0.55) < 0.6 ✓
- Complaint intent detected ✓
- Refund keyword detected ✓
- → Escalate to HITL

3

Human Review

- Agent sees: query, AI response, refund policy, customer history
- Agent notes: Customer has been loyal for 3 years, first complaint
- Agent decision: Approve refund as goodwill gesture (policy exception)

4

Feedback

- Agent feedback: "Correct decision, but should have considered customer history"
- Suggestion: "Add customer tenure to context for refund decisions"

5

Response to User

- Human-modified response: "I understand your frustration. Given your loyalty, we're happy to process a full refund..."
- User receives personalized, empathetic response

6

System Learning

- Store interaction for future training
- Update knowledge base: "Consider customer history for refund exceptions"
- Adjust confidence threshold for complaint + refund scenarios

Made with GAMMA

7. Challenges & Trade-offs

Challenge 1: Accuracy vs Speed

The core tension in RAG systems:

Accuracy focus:

- Retrieve more documents (k=10 instead of 3)
- Use larger embedding models (all-mpnet-base-v2)
- Use larger LLMs (13B instead of 7B)
- Implement multi-stage ranking
- Result: Higher quality responses, slower latency (2-5 seconds)

Speed focus:

- Retrieve fewer documents (k=1-2)
- Use lightweight embedding models (all-MiniLM-L6-v2)
- Use smaller LLMs (7B)
- Single-stage retrieval
- Result: Fast responses (< 500ms), potentially lower quality

Our choice: Balanced approach

- k=3-5 documents (sweet spot)
- all-MiniLM-L6-v2 (fast, sufficient quality)
- LLaMA 7B (real-time capable)
- Target latency: 1-2 seconds
- Acceptable for customer support (users expect quick responses)

Trade-off acceptance:

Accept occasional low-confidence responses that escalate to HITL rather than slow down all responses.

Challenge 2: Chunk Size vs Context

Chunk size directly impacts retrieval quality:

Small chunks (200 tokens):

- Pros: Precise retrieval, less noise
- Cons: Information fragmentation, context loss at boundaries
- Risk: Retrieved chunks lack sufficient context for LLM to generate coherent response

Large chunks (800+ tokens):

- Pros: Complete context, coherent information units
- Cons: Diluted relevance, slower retrieval, more tokens to LLM
- Risk: Irrelevant information in retrieved chunks confuses LLM

Our choice: 400 tokens with 50-token overlap

- Balances precision and context
- ~300 words per chunk (typical paragraph length)
- Overlap prevents boundary information loss
- Fits within LLM context window with room for multiple chunks

Trade-off acceptance:

Accept some redundancy (12.5% overlap) for improved semantic continuity.

Challenge 3: Cost vs Performance

Different component choices have cost implications:

API-based LLMs (GPT-4, Claude):

- Cost: $0.01-0.10 per query (at scale, significant)
- Performance: Highest quality responses
- Scalability: Unlimited (cloud-based)
- Privacy: Data sent to external servers

Local LLMs (LLaMA via Ollama):

- Cost: One-time download, zero per-query cost
- Performance: Good quality (slightly lower than GPT-4)
- Scalability: Limited by hardware
- Privacy: All data stays on-premises

Our choice: Local LLaMA 7B

- Cost: ~$0 per query (after initial setup)
- Performance: Acceptable for support tasks (grounded in documents)
- Scalability: Sufficient for typical support volume (100-1000 queries/day)
- Privacy: Critical for customer data

Trade-off acceptance:

Accept slightly lower response quality than GPT-4 in exchange for cost savings and privacy.

Calculation example:

```
- 1000 queries/day × 365 days = 365,000 queries/year
- GPT-4: 365,000 × $0.05 = $18,250/year
- Local LLaMA: $0/year (hardware already owned)
- Savings: $18,250/year
```

Challenge 4: Knowledge Base Completeness

RAG systems are only as good as their knowledge base:

Incomplete knowledge base:

- Queries about missing topics → Low retrieval scores
- System escalates to HITL frequently
- User frustration with "I don't know" responses

Over-complete knowledge base:

- Redundant or conflicting information
- Retrieval returns irrelevant chunks
- LLM confused by multiple perspectives

Our approach:

- Start with core documentation (FAQs, policies, common issues)
- Iteratively add content based on HITL escalations
- Regular audits to remove outdated information
- Version control for documentation changes

Trade-off acceptance:

Accept initial gaps in knowledge base; improve through feedback loop.

Challenge 5: Hallucination Risk

Even with RAG, LLMs can hallucinate:

Hallucination scenarios:

- Retrieved documents don't contain answer, LLM generates plausible-sounding response
- LLM misinterprets retrieved context
- LLM combines information from multiple chunks incorrectly

Mitigation strategies:

1. Explicit instruction in prompt: "Answer ONLY based on provided context"
2. Confidence scoring: Flag responses with low retrieval scores
3. HITL escalation: Low-confidence responses reviewed by humans
4. Temperature setting: Use temperature=0.3 (low randomness)
5. Response validation: Check for uncertainty phrases

Trade-off acceptance:

Accept that some hallucinations will occur; catch them through HITL.

Challenge 6: Intent Classification Ambiguity

Some queries don't fit neatly into intent categories:

Ambiguous query example:

> "I was charged $50 but I only used the service for 1 day"
>
> Could be: Billing (overcharge), Account (usage tracking), Technical (billing system error)

Consequences:

- Wrong intent → Wrong document retrieval → Wrong response
- Ambiguous intent → Uncertain routing → Escalation

Our approach:

- Multi-intent support: Allow queries to have primary + secondary intents
- Uncertain routing: Retrieve from multiple intent categories when confidence < 0.7
- Confidence weighting: Lower confidence for ambiguous intents
- HITL escalation: Ambiguous queries more likely to escalate

Trade-off acceptance:

Accept that some queries will be ambiguous; handle through broader retrieval and escalation.

Challenge 7: Latency Under Load

System performance degrades with high query volume:

Bottlenecks:

- Embedding generation: ~10ms per query
- Vector database search: ~50ms for large collections
- LLM inference: 500ms-2s depending on response length
- Total: 1-3 seconds per query

At scale (1000 queries/hour):

- Single machine may not handle load
- Response times increase
- User experience degrades

Solutions:

1. Caching: Cache embeddings for common queries
2. Batching: Process multiple queries in parallel
3. Scaling: Distribute across multiple machines
4. Optimization: Use quantized models (faster inference)

Trade-off acceptance:

Accept that scaling requires infrastructure investment; start with single machine, scale as needed.

Summary of Key Trade-offs

Trade-off	Choice	Rationale
Accuracy vs Speed	Balanced (1-2s latency)	Customer support needs quick responses
Chunk Size	400 tokens	Sweet spot between precision and context
Cost vs Performance	Local LLaMA	Privacy + cost savings > slightly lower quality
Knowledge Base	Iterative growth	Start small, improve through feedback
Hallucination Risk	Mitigate + escalate	Accept some risk, catch through HITL
Intent Ambiguity	Multi-intent + escalation	Handle complexity through broader retrieval
Scalability	Single machine initially	Scale when needed, not before

Made with GAMMA

8. Testing Strategy

Our comprehensive testing strategy ensures the reliability, performance, and accuracy of the RAG system across various scenarios. We employ a multi-faceted approach, covering different stages of development and deployment.

Test Categories

01
Unit Tests
Individual components

02
Integration Tests
Component interactions

03
End-to-End Tests
Full workflow

04
Performance Tests
Latency and throughput

05
User Acceptance Tests
Real-world scenarios

Sample Test Queries & Expected Outputs

To validate the system's behavior, we've defined specific test sets with predetermined queries, expected system responses, and pass criteria.

Test Set 1: Billing Intent (High Confidence)

> Query 1: "How do I get a refund?"

- Expected intent: Billing
- Expected retrieval: Refund policy document
- Expected confidence: > 0.8
- Expected response: Clear refund process with timeline
- Test result: PASS if response mentions 30-day window and process steps

> Query 2: "Why was I charged twice?"

- Expected intent: Billing
- Expected retrieval: Billing FAQ, duplicate charge section
- Expected confidence: > 0.75
- Expected response: Explanation of duplicate charges and resolution steps
- Test result: PASS if response offers investigation or refund option

> Query 3: "Can I change my subscription plan?"

- Expected intent: Billing
- Expected retrieval: Subscription management document
- Expected confidence: > 0.8
- Expected response: Steps to change plan or link to account settings
- Test result: PASS if response is actionable and accurate

Test Set 2: Technical Intent (Medium Confidence)

> Query 4: "The app crashes when I upload files"

- Expected intent: Technical
- Expected retrieval: Known issues, file upload troubleshooting
- Expected confidence: 0.65–0.75
- Expected response: Troubleshooting steps (clear cache, check file size, etc.)
- Test result: PASS if response provides actionable debugging steps

> Query 5: "API returns 500 error"

- Expected intent: Technical
- Expected retrieval: API documentation, error codes
- Expected confidence: 0.7–0.8
- Expected response: Explanation of 500 error and common causes
- Test result: PASS if response mentions server issues and retry strategy

> Query 6: "How do I integrate with Slack?"

- Expected intent: Technical
- Expected retrieval: Integration guide, API documentation
- Expected confidence: 0.75–0.85
- Expected response: Step-by-step integration instructions
- Test result: PASS if response includes authentication and webhook setup

Test Set 3: Account Intent (High Confidence)

> Query 7: "I forgot my password"

- Expected intent: Account
- Expected retrieval: Password reset guide
- Expected confidence: > 0.85
- Expected response: Clear password reset instructions
- Test result: PASS if response includes link to reset page

> Query 8: "How do I update my profile?"

- Expected intent: Account
- Expected retrieval: Account settings documentation
- Expected confidence: > 0.8
- Expected response: Navigation steps to profile settings
- Test result: PASS if response is clear and actionable

Test Set 4: Ambiguous Intent (Low-Medium Confidence)

> Query 9: "I can't log in and I'm being charged"

- Expected intent: Account (primary), Billing (secondary)
- Expected confidence: 0.65–0.75 (ambiguous)
- Expected behavior: Retrieve from both categories
- Expected response: Address both login and billing issues
- Test result: PASS if response covers both concerns

> Query 10: "The app is slow and expensive"

- Expected intent: Technical (primary), Billing (secondary)
- Expected confidence: 0.60–0.70 (ambiguous)
- Expected behavior: Retrieve from both categories
- Expected response: Performance tips and pricing explanation
- Test result: PASS if response addresses both concerns

Test Set 5: Out-of-Scope Queries (Low Confidence → Escalation)

> Query 11: "What's your company's environmental policy?"

- Expected intent: Unknown
- Expected retrieval: No relevant documents (score < 0.3)
- Expected confidence: < 0.6
- Expected behavior: Escalate to HITL
- Test result: PASS if query escalates without generating hallucinated response

> Query 12: "Can you help me with a competitor's product?"

- Expected intent: Unknown
- Expected retrieval: No relevant documents
- Expected confidence: < 0.6
- Expected behavior: Escalate to HITL
- Test result: PASS if system admits knowledge gap

> Query 13: "What's the weather today?"

- Expected intent: Unknown
- Expected retrieval: No relevant documents
- Expected confidence: < 0.6
- Expected behavior: Escalate to HITL
- Test result: PASS if system doesn't hallucinate weather information

Test Set 6: Complaint Intent (Automatic Escalation)

> Query 14: "Your service is terrible and I want a refund!"

- Expected intent: Complaint + Billing
- Expected behavior: Automatic escalation (complaint intent)
- Expected confidence: Irrelevant (escalates regardless)
- Expected response: Escalation to human agent
- Test result: PASS if escalated immediately without AI response

> Query 15: "I've been a customer for 5 years and this is unacceptable"

- Expected intent: Complaint
- Expected behavior: Automatic escalation
- Expected response: Escalation to human agent
- Test result: PASS if escalated with high priority

HITL Scenario Tests

Human-in-the-Loop (HITL) scenarios are crucial for evaluating the system's ability to integrate human oversight and feedback.

Scenario 1: AI Response Approved by Human

Setup:

- Query: "How do I cancel my subscription?"
- AI response: "You can cancel anytime from account settings → Billing → Cancel Subscription"
- Confidence: 0.72 (escalated due to refund keyword)

Human review:

- Agent reads query and AI response
- Agent checks knowledge base: Response is accurate
- Agent decision: Approve AI response
- Feedback: "Correct, clear, and actionable"

Expected outcome:

- Response sent to user with note: "Reviewed by support team"
- Interaction logged for analytics
- No system improvement needed

Scenario 2: AI Response Modified by Human

Setup:

- Query: "I was overcharged. What should I do?"
- AI response: "According to our policy, refunds are available within 30 days of purchase"
- Confidence: 0.58 (low due to ambiguity)

Human review:

- Agent reads query and AI response
- Agent checks customer history: Customer has been loyal for 3 years
- Agent notes: Policy is correct, but response lacks empathy
- Agent decision: Modify response to include apology and goodwill gesture

Modified response:

> "I sincerely apologize for the overcharge. Given your loyalty, we'd like to process a full refund immediately. Here's how..."

Expected outcome:

- Modified response sent to user
- Interaction logged with feedback: "Correct but needed empathy"
- System learning: Add customer history context for refund decisions

Scenario 3: AI Response Rejected, Custom Response Provided

Setup:

- Query: "Can you make an exception to your refund policy?"
- AI response: "Our refund policy allows refunds within 30 days. I cannot make exceptions."
- Confidence: 0.55 (low due to policy exception request)

Human review:

- Agent reads query and AI response
- Agent checks customer history: New customer, first purchase
- Agent checks policy: Exceptions require manager approval
- Agent decision: Reject AI response, escalate to manager

Manager review:

- Manager approves exception based on customer circumstances
- Manager provides custom response: "We understand your situation. We're happy to make an exception..."

Expected outcome:

- Custom response sent to user
- Interaction logged with feedback: "Correct escalation, exception approved"
- System learning: Add exception approval workflow

Performance Test Metrics

Key performance indicators (KPIs) are established to monitor system efficiency and responsiveness.

Latency benchmarks:

- Embedding generation: < 20ms
- Vector search: < 100ms
- LLM inference: < 2000ms
- Total end-to-end: < 2500ms (2.5 seconds)

Throughput benchmarks:

- Single machine: 100-200 queries/minute
- Acceptable for typical support volume

Accuracy metrics:

- Retrieval precision: > 80% (top-3 documents contain relevant info)
- Intent classification accuracy: > 85%
- Response quality (human evaluation): > 80% acceptable

Regression Testing

After system updates, a regression test protocol is followed to ensure no new issues are introduced.

- Re-run all sample queries
- Verify confidence scores haven't degraded
- Check that escalation rates remain in target range (15-25%)
- Validate HITL workflow still functions correctly

Continuous Testing

Ongoing monitoring and analysis are critical for maintaining system health and identifying areas for improvement.

- Track escalation rate daily
- Monitor human approval rate
- Analyze failed queries for patterns
- Identify knowledge base gaps
- Adjust thresholds based on metrics

Made with GRAMMARLY

9. Future Enhancements & Roadmap

Phase 1: Multi-Document Support (Q2 2026)

Current limitation: System handles single knowledge base

Enhancement:

- Support multiple document sources (FAQs, API docs, blog posts, video transcripts)
- Metadata tagging: Document type, version, date, author
- Source-specific retrieval: Prioritize certain sources for specific intents
- Document versioning: Track changes and maintain historical context

Implementation:

- Extend ChromaDB metadata to include document source
- Add source filtering in retrieval node
- Create document ingestion pipeline for multiple formats
- Implement document update workflow

Benefits:

- Richer knowledge base
- Better context for complex queries
- Ability to cite specific sources
- Support for evolving documentation

Phase 2: Feedback Loop & Active Learning (Q3 2026)

Current limitation: System doesn't learn from HITL feedback in real-time

Enhancement:

- Collect human feedback systematically
- Use feedback to improve confidence thresholds
- Identify knowledge base gaps
- Retrain intent classifier with new examples
- Fine-tune LLM prompts based on feedback patterns

Implementation:

- Build feedback collection UI for HITL agents
- Create feedback analysis pipeline
- Implement threshold adjustment algorithm
- Set up automated retraining workflow
- A/B test prompt variations

Metrics:

- Track improvement in escalation rate over time
- Monitor human approval rate trends
- Measure knowledge base gap closure rate
- Evaluate prompt effectiveness

Benefits:

- Continuous system improvement
- Reduced escalation rate over time
- Better intent classification
- More effective prompts

Phase 3: Conversation Memory & Context (Q4 2026)

Current limitation: System treats each query independently

Enhancement:

- Maintain conversation history across multiple turns
- Use conversation context to disambiguate queries
- Track user preferences and history
- Implement context-aware response generation
- Support follow-up questions without repetition

Implementation:

- Extend state to include full conversation history
- Add context summarization node (compress long histories)
- Implement context-aware retrieval (use conversation history to refine queries)
- Add user profile integration (preferences, history)
- Create follow-up question detection

Example:

- User: "How do I reset my password?"
- System: [Provides password reset instructions]
- User: "I didn't receive the email"
- System: [Understands context: password reset email issue, not general password reset]
- System: [Retrieves troubleshooting for password reset emails]

Benefits:

- More natural conversations
- Better understanding of user intent
- Reduced repetition
- Improved user satisfaction

Phase 4: Multi-Language Support (Q1 2027)

Current limitation: System only supports English

Enhancement:

- Support multiple languages (Spanish, French, German, Mandarin, etc.)
- Automatic language detection
- Language-specific embeddings
- Translated knowledge base
- Multilingual intent classification

Implementation:

- Use multilingual embedding model (e.g., multilingual-e5)
- Implement language detection in input node
- Create translated versions of knowledge base
- Build multilingual intent classifier
- Add language preference to user profile

Challenges:

- Knowledge base translation (manual or automated)
- Language-specific nuances in intent detection
- Increased storage for multilingual embeddings

Benefits:

- Global customer support
- Reduced need for human translators
- Better user experience for non-English speakers

Phase 5: Advanced Routing & Specialization (Q2 2027)

Current limitation: Single LLM handles all query types

Enhancement:

- Specialized LLMs for different domains (billing, technical, account)
- Dynamic routing based on intent
- Expert system integration for complex logic
- Rule-based routing for high-confidence scenarios

Implementation:

- Fine-tune separate LLMs for each intent category
- Implement intent-based routing in generation node
- Create rule engine for deterministic responses
- Add expert system for policy decisions

Example:

- Billing query → Route to fine-tuned billing LLM
- Technical query → Route to fine-tuned technical LLM
- Policy exception → Route to rule engine + human review

Benefits:

- Higher quality responses for specific domains
- Faster responses for deterministic queries
- Better handling of domain-specific nuances

Phase 6: Proactive Support & Recommendations (Q3 2027)

Current limitation: System only responds to user queries

Enhancement:

- Analyze user behavior to identify potential issues
- Proactive recommendations based on usage patterns
- Predictive escalation (identify issues before user reports)
- Personalized help suggestions

Implementation:

- Integrate with user activity logs
- Build behavior analysis pipeline
- Create recommendation engine
- Implement proactive notification system

Example:

- User hasn't logged in for 30 days → Suggest account recovery
- User has high API error rate → Suggest troubleshooting
- User approaching billing limit → Suggest plan upgrade

Benefits:

- Improved user retention
- Reduced support tickets
- Better user experience
- Increased customer satisfaction

Phase 7: Integration with External Systems (Q4 2027)

Current limitation: System is isolated from business systems

Enhancement:

- Integration with CRM (customer history, preferences)
- Integration with ticketing system (create tickets from escalations)
- Integration with knowledge management system (auto-update docs)
- Integration with analytics platform (track metrics)
- Integration with payment system (process refunds)

Implementation:

- Build API connectors for each system
- Implement secure authentication
- Create data sync pipelines
- Add transaction logging

Benefits:

- Seamless customer experience
- Reduced manual work for agents
- Better data consistency
- Improved operational efficiency

Phase 8: Deployment & Scaling (Q1 2028)

Current limitation: Single-machine deployment

Enhancement:

- Containerized deployment (Docker)
- Kubernetes orchestration for scaling
- Load balancing across multiple instances
- Database replication for high availability
- Monitoring and alerting

Implementation:

- Create Docker images for all components
- Set up Kubernetes cluster
- Implement load balancer
- Configure database replication
- Build monitoring dashboard

Scaling targets:

- Support 10,000+ queries/day
- 99.9% uptime SLA
- < 2 second response time at scale

Benefits:

- Enterprise-grade reliability
- Ability to handle growth
- Reduced operational overhead
- Better disaster recovery

Technology Roadmap Summary

Phase	Timeline	Focus	Key Deliverables
1	Q2 2026	Multi-document support	Document ingestion, source filtering
2	Q3 2026	Feedback loop	Feedback collection, threshold tuning
3	Q4 2026	Conversation memory	Context tracking, follow-up handling
4	Q1 2027	Multi-language	Language detection, translation
5	Q2 2027	Specialization	Domain-specific LLMs, routing
6	Q3 2027	Proactive support	Behavior analysis, recommendations
7	Q4 2027	External integrations	CRM, ticketing, payment systems
8	Q1 2028	Enterprise deployment	Kubernetes, scaling, monitoring

Success Metrics for Future Phases

- Escalation rate: Target < 10% (from current 15-25%)
- Human approval rate: Target > 85% (from current 60-70%)
- Response quality: Target > 90% acceptable (from current 80%)
- System uptime: Target 99.9%
- Average response time: Target < 1.5 seconds
- Customer satisfaction: Target > 4.5/5 stars
- Cost per query: Target < $0.001 (including infrastructure)

Made with GRAMMARLY

Conclusion: Building Intelligent Customer Support

Summary

This technical documentation outlines a comprehensive RAG-based customer support assistant that combines:

- **Retrieval-Augmented Generation (RAG)**: Grounding responses in actual company documentation to eliminate hallucinations
- **LangGraph Orchestration**: Stateful workflow management with conditional routing and escalation logic
- **Human-In-The-Loop (HITL)**: Strategic human oversight for complex queries and continuous system improvement
- **Intelligent Confidence Scoring**: Multi-factor evaluation to determine when to escalate vs. respond

The system achieves a balance between automation and human judgment, delivering fast, accurate responses while maintaining quality through strategic escalation.

Key Achievements

Accuracy
Responses grounded in company documentation (no hallucinations)

Speed
1-2 second response times suitable for real-time support

Cost-Effective
Local LLM deployment eliminates per-query API costs

Privacy
All customer data remains on-premises

Scalability
Designed for growth from 100 to 10,000+ queries/day

Transparency
Clear escalation logic and human oversight

Continuous Improvement
Feedback loop enables system learning

Critical Success Factors

1 — Knowledge Base Quality

System is only as good as its documentation

- Start with core FAQs and policies
- Iteratively expand based on HITL feedback
- Regular audits to maintain accuracy

2 — Confidence Threshold Tuning

Balance between automation and escalation

- Too high: Escalate too many queries (defeats automation)
- Too low: Miss errors and escalate too few (quality issues)
- Target: 15-25% escalation rate

3 — HITL Workflow Efficiency

Human agents must be empowered to improve the system

- Provide clear feedback mechanisms
- Act on feedback systematically
- Track improvement metrics

4 — Monitoring & Metrics

Continuous observation of system health

- Track escalation rate, approval rate, response quality
- Identify patterns in failures
- Adjust thresholds and prompts based on data

Implementation Recommendations

1 — Phase 1 (Immediate)

- Deploy core RAG system with LangGraph orchestration
- Implement HITL workflow with feedback collection
- Establish monitoring and metrics dashboard
- Train support team on HITL interface

2 — Phase 2 (1-3 months)

- Expand knowledge base based on HITL feedback
- Fine-tune confidence thresholds
- Optimize prompt design
- Implement feedback loop for continuous improvement

3 — Phase 3 (3-6 months)

- Add multi-document support
- Implement conversation memory
- Expand to additional intents
- Scale infrastructure as needed

Challenges to Anticipate

Knowledge Base Gaps

Initial system will have incomplete documentation

- **Mitigation**: Expect high escalation rate initially (30-40%)
- Plan for rapid knowledge base expansion
- Use HITL feedback to identify gaps

Intent Ambiguity

Some queries don't fit neatly into categories

- **Mitigation**: Use multi-intent support and broader retrieval
- Accept that some queries will escalate
- Improve intent classifier over time

Hallucination Risk

LLMs can still generate plausible-sounding errors

- **Mitigation**: Explicit prompts, confidence scoring, HITL escalation
- Monitor for hallucination patterns
- Adjust thresholds if hallucinations increase

Scaling Challenges

Single-machine deployment has limits

- **Mitigation**: Plan for containerization and Kubernetes early
- Monitor latency as query volume increases
- Scale proactively before hitting limits

Measuring Success

Track these metrics over time:

- **Month 1-3:**
  - Escalation rate: 25-35% (expected high initially)
  - Human approval rate: 50-60% (system learning)
  - Response quality: 70-75% acceptable
  - Average response time: 1-2 seconds
- **Month 3-6:**
  - Escalation rate: 15-25% (improving)
  - Human approval rate: 65-75% (system improving)
  - Response quality: 80-85% acceptable
  - Average response time: < 1.5 seconds
- **Month 6+:**
  - Escalation rate: < 15% (optimized)
  - Human approval rate: > 80% (highly accurate)
  - Response quality: > 85% acceptable
  - Average response time: < 1 second

Final Thoughts

This RAG-based customer support assistant represents a pragmatic approach to AI-powered support. Rather than attempting to build a perfect autonomous system, it acknowledges human expertise and integrates it strategically through HITL workflows.

The system's strength lies not in eliminating human agents, but in augmenting them—handling routine queries automatically while escalating complex issues to humans who can provide nuanced, empathetic responses.

By combining retrieval-augmented generation, intelligent routing, and human oversight, this system delivers the best of both worlds: the speed and consistency of automation with the judgment and empathy of human support.

The roadmap ensures continuous evolution, with planned enhancements for multi-language support, conversation memory, proactive recommendations, and enterprise-scale deployment.

Success requires commitment to:

- Building and maintaining a quality knowledge base
- Empowering human agents through effective HITL workflows
- Monitoring system performance and acting on insights
- Iterating based on feedback and metrics

With these foundations in place, this system can transform customer support from a cost center into a competitive advantage.

Made with GAMMA

Made with GAMMA